

Sharif University of Technology
Department of Electrical Engineering



Data Networks

Final Project Report

BitTorrent, A peer to peer file sharing system

Mohammadreza Darvish

402101663

Course Professor: Dr. Pakravan

Semester: Second Semester, 1404-1405

Contents

1. Introduction
2. Scope, Environment, and Assumptions
3. System Architecture
4. Implementation Flow
5. Metainfo and Torrent Identity
6. Tracker Design
7. Peer Design and Peer-Wire Protocol
8. Piece Management and File Integrity
9. Logging and Observability
10. Wireshark Capture and TCP Interpretation
11. End-to-End Verification Results
12. Design Decisions
13. Conclusion

1. Introduction

This project implements a BitTorrent system. A multithreaded HTTP tracker groups peers by torrent info hash and returns peer lists. Each peer then uses direct TCP connections to perform a handshake, exchange availability information, request blocks, verify complete pieces with SHA-1, and store verified data.

The implementation is written in Python and uses only the Python standard library at runtime. It includes a custom bencoder/decoder, metainfo handling, a tracker, peer-wire message framing, piece and file management, periodic tracker contact, keep-alive messages, a torrent thread pool, and structured event logs.

Three recorded executions were included. The first transferred a 48,932-byte JPEG between two peers. The second used three peers with a different torrent: one peer downloaded a text file, the original seeder stopped, and a third peer then downloaded from the former downloader. The third used alphabet.torrent to transfer a 26-byte file as three pieces of 10, 10, and 6 bytes. Byte-for-byte comparison confirmed that the received files matched their sources.

2. Scope, Environment, and Assumptions

2.1 Deployment scope

The system was deployed on one Ubuntu virtual machine and operated through several terminal sessions. The tracker and all peers therefore used the IPv4 loopback address 127.0.0.1. This is permitted by the specification, which allows all components to run on the same device. Separate listening ports identify the services even though their IP address is the same.

Endpoint	Bind/address	Port	Purpose
Tracker service	0.0.0.0; reached as 127.0.0.1	6969	HTTP /announce endpoint
Peer services	0.0.0.0; reached as 127.0.0.1	6881-6887 in demonstrations	Incoming peer-wire TCP connections
HTTP client sockets	127.0.0.1	OS-selected ephemeral port, e.g. 33480	Temporary connection from a peer to tracker port 6969
Peer TCP client sockets	127.0.0.1	OS-selected ephemeral port, e.g. 58474	Temporary outgoing side of a connection to a peer listener

Table 1 - Addressing and port roles in the localhost deployment.

2.2 Explicit assumptions

- Peers possess the same .torrent metainfo before joining a swarm.
- The tracker and peers operate on localhost or on different VMs within the same IP range. For a multi-VM deployment, the metainfo must contain a tracker address reachable from those peers. NAT traversal, port forwarding, and firewall configuration are outside the network model used here.
- Tracker swarm state is kept in memory. Restarting the tracker clears the current registrations. Peers that were already running must then be restarted or explicitly send another started announce before they are registered in the new tracker state.
- Piece integrity and torrent identity use SHA-1 with 20-byte digests.

2.3 Runtime Dependencies

The tracker and peer processes use Python standard-library modules for their runtime functionality. http.server and urllib handle tracker HTTP communication, socket is used for peer TCP connections, and threading provides the concurrency needed by the tracker and peer sessions. hashlib is used for SHA-1 calculations, while struct handles binary protocol fields and pathlib manages file-system paths. The implementation also uses datetime, time, secrets, and sys for timestamps, timers, peer-ID generation, and process-level support.

The bencoding and bencoding functions are implemented inside common/bencode.py, so metainfo and tracker data pass through the same local encoding and decoding logic used by the rest of the system.

2.4 Functional Scope

The implementation covers tracker-assisted peer discovery, direct peer-to-peer TCP communication, piece verification and storage, concurrent torrent sessions, logging, and packet observation. Metainfo identifies the torrent and provides the tracker information needed to join its swarm, while file data is exchanged directly between peers. The detailed component structure and runtime sequence are described in the System Architecture and Implementation Flow sections.

3. System Architecture

The architecture separates coordination from data transfer. The tracker never carries file data; it only registers peers and returns endpoints for a selected info hash. Once discovery is complete, peers exchange control messages and file blocks directly over TCP.

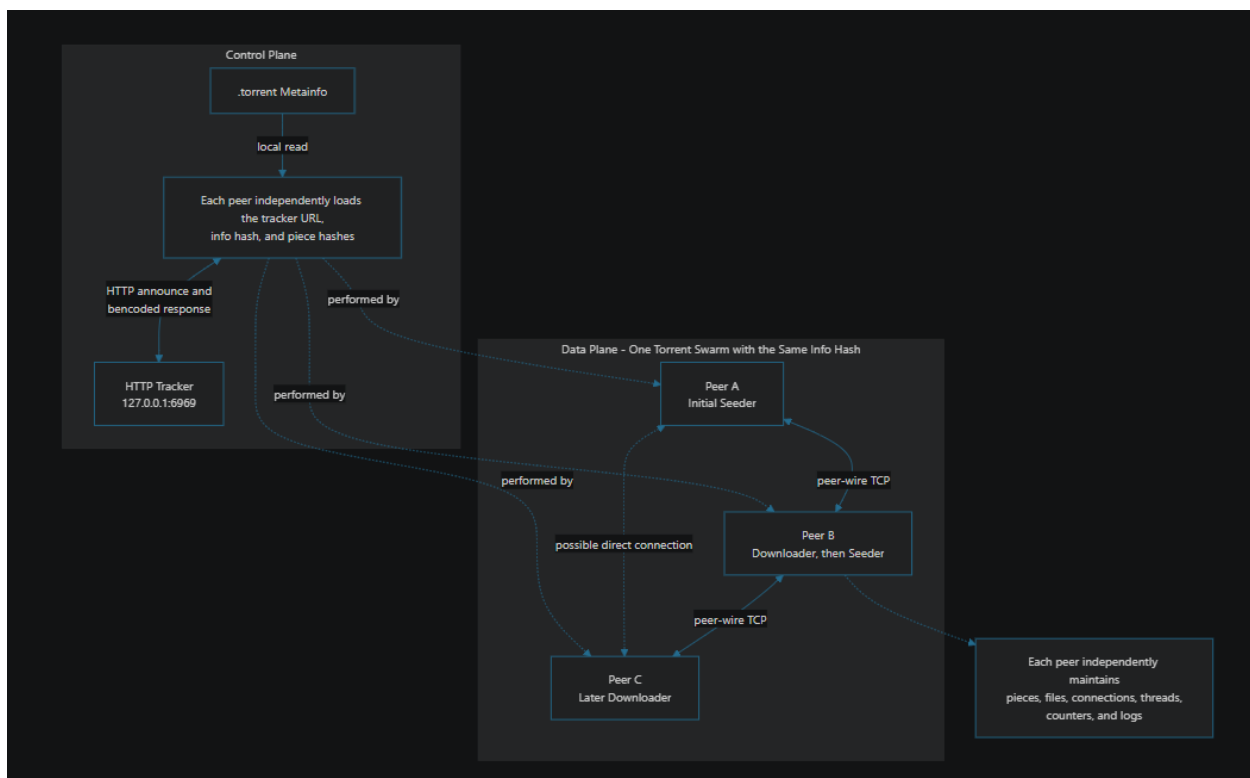


Figure 1 - System context and communication paths.

3.1 Component responsibilities

- **common/bencode.py.** Encodes and decodes deterministic bencoded integers, byte strings, lists, and dictionaries.
- **common/metainfo.py.** Loads and validates metainfo, calculates torrent identity and total size, and provides the tracker URL.
- **common/protocol.py.** Centralizes handshake sizes and peer-wire message identifiers shared by the network modules.

- **common/logger.py.** Serializes structured event logging and maintains the required creation and last-modified header fields.
- **tracker/tracker.py.** Serves /announce concurrently and maintains independent in-memory swarms keyed by info hash.
- **peer/messages.py.** Builds and parses the binary handshake and length-prefixed peer-wire messages.
- **peer/piece_manager.py.** Tracks verified availability, checks hashes, maps logical data ranges, and reads or writes torrent files safely.
- **peer/peer.py.** Coordinates tracker access, TCP sessions, connection state, transfer decisions, timers, and torrent pools.
- **torrent_generator.py.** Creates single-file metainfo from shared_files by dividing data into pieces and calculating SHA-1 hashes.

This division keeps binary encoding, metainfo interpretation, logging, and file integrity separate from network orchestration. The peer coordinator is necessarily the largest module because it owns the live state machine, but message construction and piece storage remain isolated so their correctness does not depend on connection-management details.

3.2 State and concurrency model

Tracker state is organized hierarchically by swarm, then peer ID, then peer record. Each peer record contains the peer's IP address, listening port, uploaded/downloaded/left counters, and last-seen time. A lock protects this shared structure while the threaded HTTP server processes simultaneous announces.

A peer session owns one torrent, one listening socket, one piece manager, its tracker counters, and its active connection set. Background work is divided among an accept loop, periodic tracker refresh, periodic keep-alive handling, and per-connection receive loops. Locks protect shared counters, piece reservations, connection lists, and socket writes. A torrent pool starts one independent session for each configured torrent.

4. Implementation Flow

Establishing the implementation order. We did not start by trying to build the complete peer at once. The first step was to turn the required BitTorrent workflow into an implementation order with clear dependencies. The smallest useful path was metainfo and binary encoding first, then tracker registration and discovery, then a TCP connection between peers, and only after that the availability, request, transfer, verification, and seeding behavior. This order mattered because most later stages depend on bytes produced by the earlier ones. For example, a peer cannot calculate a stable torrent identity until the info dictionary is encoded deterministically, and peer-to-peer transfer is not useful until the tracker can return a reachable endpoint. We kept the tracker and peer roles separate from the beginning: the tracker would only coordinate swarm membership, while the actual file bytes would move on direct TCP connections. The localhost Ubuntu setup also let us develop the protocol behavior without mixing it with NAT or firewall problems that were outside the project scope.

Building the encoding layer first. The first module we implemented was `common/bencode.py` because both metainfo processing and tracker responses depend on Bencode. The encoder was kept deterministic, especially for dictionaries, whose keys are emitted in sorted byte order. This became important later when the info hash was calculated from the encoded info dictionary; two peers must obtain the same 20-byte identity from the same metainfo. The decoder was written as a recursive parser that advances through a byte buffer and returns the next unread position together with the decoded value. Before connecting any sockets, we exercised this layer with normal and malformed values so that later tracker failures would not be mixed with basic encoding errors. This also exposed small language-level issues early, such as Python Boolean values behaving as integers even though Bencode has no Boolean type. By resolving these cases at the encoding boundary, the networking code could treat Bencode as a stable service instead of repeatedly handling format exceptions inside HTTP and TCP logic.

Bringing up the tracker before the full peer. Once the encoding path was stable, we moved to the tracker. The first design separated HTTP handling and swarm state into two files, but the amount of state was small and tightly coupled to announce processing, so we combined those responsibilities in `tracker/tracker.py`. The tracker was then brought up as a `ThreadingHTTPServer` on port 6969 with an in-memory structure organized by `info_hash` and then `peer_id`. Each handler could process an announce concurrently, so access to the shared swarm dictionary was protected with a lock. At this stage the goal was not yet file transfer; it was to make registration, peer discovery, and lifecycle updates predictable. We parsed the binary identifiers from percent-encoded query data rather than treating them as normal text, and we kept the peer's advertised listening port separate from the temporary source port of the HTTP connection. The final request and response fields are described in Section 6; during implementation, the main work was making the state transitions behind those fields consistent.

Correcting tracker lifecycle behavior. The first tracker version exposed a lifecycle problem: a peer that had never registered could send an ordinary update without a started event and still enter the swarm. That made the tracker state convenient for testing but incorrect for the intended protocol flow. We changed the announce logic so an unknown peer must begin with `event=started`, while an already registered peer can later refresh with an omitted or empty event. Seeder and leecher accounting was also tied directly to the left counter, rather than to assumptions about how a peer entered the swarm. Timeout handling was another point that needed adjustment. We used monotonic time for last-seen aging so wall-clock corrections would not affect peer expiry. The normal timeout was 90 seconds with a 30-second announce interval, but during development we temporarily reduced the timeout to three seconds so stale-peer removal could be checked without waiting through full intervals. After that behavior was stable, the normal values were restored. Cleanup was placed before response construction so an expired peer would not be returned in the next list.

Adding logging while the system was still small. After the tracker was functioning, we added `common/logger.py` before the peer became highly concurrent. This was a practical debugging decision. Once tracker handlers, peer receiver loops, keep-alive activity, and file transfer are running at the same time, terminal output alone does not preserve enough ordering to reconstruct what happened. The logger therefore became part of the development path rather than a feature added at the end. Tracker activity was written to one file and each peer used a port-specific log, with a process-wide lock preventing concurrent threads from interleaving event records or header updates. We kept the log

entries at the protocol-event level instead of dumping raw socket data. That made it possible to follow registration, connection, handshake, interest state, requests, piece verification, completion, and shutdown chronologically while Wireshark was later used for the actual bytes on the wire. The final log structure and event categories are covered in Section 9.

Moving from discovery to a real TCP session. With tracker discovery available, we defined the fixed peer-wire constants in `common/protocol.py` and the message construction and parsing rules in `peer/messages.py`. We then built only the first part of `peer/peer.py`: load the torrent identity, generate a peer ID, create the HTTP announce, decode the tracker response, extract peer endpoints, and open a listening socket. The listener was deliberately started before the started announce. Otherwise, another peer could receive the advertised endpoint during the short interval in which no process was yet accepting connections. Two peers on ports 6881 and 6882 were enough to check this stage: the first initially saw no other endpoint, and after the second registered the tracker returned the opposite peer. Only after that path worked did we add the 68-byte BitTorrent handshake. This separated two different failure classes: if peers could not discover each other, the problem was tracker-side; if they discovered each other but could not establish a session, the problem was in the TCP or handshake path.

Treating TCP as a byte stream. The next issue was message framing. A single `recv` call cannot be assumed to return one complete handshake or one complete BitTorrent message, because TCP preserves byte order but not application-message boundaries. We therefore added `recv_exact` and used it first for the 68-byte handshake and then for normal peer-wire frames. The receiver reads exactly four bytes for the length prefix, decodes the declared body length, and continues reading until that body is complete. A zero length is handled as a keep-alive without a message ID. This implementation choice was made before the larger transfer state machine, because every later message depends on reliable framing. It also made the code consistent with what we later observed in Wireshark: one logical message can be divided across TCP segments, and several small messages can appear close together in the same stream. Section 7 describes the final message formats, while Section 10 shows why packet boundaries cannot be used as protocol boundaries.

Introducing per-connection state before data transfer. After handshakes worked in both directions, we added a separate state object for every peer connection. The local and remote choking states, local and remote interest states, remote availability, current piece reservation, outstanding block request, connection status, and socket send lock all had to belong to the connection rather than to the peer process globally. This became especially important because BitTorrent flow control is bidirectional: being allowed to download from a remote peer does not imply that the remote peer is allowed to download in the opposite direction. We then added `BITFIELD` and `HAVE` handling and used the remote availability set to decide whether `INTERESTED` or `NOT_INTERESTED` should be sent. One intermediate bug appeared in the unchoke path: the `UNCHOKE` frame reached the remote side while the stored local state still indicated that the peer was choking. We corrected the order so the state is changed before the frame is sent, with rollback if the send fails. That removed a mismatch between protocol bytes and the connection state used by later request checks.

Separating piece integrity from network orchestration. Once the control exchange was stable, the next dependency was not another socket feature but a reliable piece manager. We created `peer/piece_manager.py` so `peer.py` would not have to combine connection state, file layout, hashing, and disk writes in one large path. The piece manager treats torrent content as one logical byte range

and maps it to either a single file or the ordered multi-file layout. On startup it scans the existing data and marks a piece available only when its SHA-1 digest matches the metainfo. That same verified set is used to construct the local bitfield, so a file's mere presence is not enough to advertise it. Storage validation was added at the same stage because metainfo paths eventually become file-system paths. Absolute paths, parent traversal, embedded separators, null bytes, and paths that resolve outside the selected data root are rejected. A newly received piece is also verified before it is committed, then read back from storage for a second check. The detailed integrity and mapping rules are kept in Section 8; in the implementation sequence, the important point is that this subsystem was working before network data was allowed to modify files.

Integrating block transfer with piece verification. After the piece manager could independently read, validate, and write torrent ranges, we connected it to the peer-wire state machine. A missing piece is first reserved for one connection so two local receiver threads do not request the same work at the same time. We used a simple deterministic selection policy and transferred each piece through blocks of at most 16 KiB, with only one outstanding block request per connection. This was slower than a pipelined design but kept request validation unambiguous while the protocol was being built. When a PIECE frame arrives, its piece index, begin offset, and block length must match the outstanding request before the bytes are appended to the in-memory piece buffer. If the completed piece fails its SHA-1 check, it is not written or advertised and the reservation is released for another attempt. If it passes, left is recalculated, the data is saved, HAVE is sent, and the next useful piece can be selected. When left reaches zero, completed is announced once, but the peer remains active so the downloaded data can be uploaded to later peers.

Making upload and download use the same verified state. The upload path was added immediately after successful downloading so a completed peer could transition into a seeder without a separate code path. A REQUEST is served only when the connection state permits it, the requested range is valid, and the piece is still present in the verified availability set. We also re-check the piece before serving its bytes. This matters because startup verification only describes the file at startup; if the stored content changes afterward, the peer should not continue advertising and uploading corrupted data. The uploaded byte counter is increased only for block bytes that were actually sent. This linked the transfer logic back to the tracker counters and completed state without treating completion as the end of the peer lifecycle. In the later three-peer run, this behavior was what allowed a downloader to remain in the swarm and become the source for a new peer after the original seeder stopped.

Expanding concurrency only after the single-session path worked. Once one peer connection could discover, handshake, request, verify, and serve data, we expanded the peer into its final concurrent form. A session now has an accept loop, periodic tracker refresh, a keep-alive loop, one receive loop per connected peer, and the main lifecycle path. Separate locks protect connection collections, tracker activity, piece reservations, counters, piece availability, and socket writes. The tracker update interval and the 15-second peer keep-alive were kept as independent mechanisms because they operate on different connections and solve different liveness problems. This stage also exposed a shutdown race. Early runs sometimes logged Bad file descriptor when one thread closed a socket while another receive loop was still blocked on it. We changed shutdown ordering so the running state and stop event are updated first, connections are marked inactive, reservations are released, expected OSError conditions

are handled, sockets are then closed, and worker threads are joined briefly. Normal shutdown could then send stopped and exit without reporting an expected close as a protocol failure.

Generalizing the working peer to multiple torrents. After the single-torrent session was stable, we implemented the torrent pool as a collection of complete peer sessions rather than as a generic pool of worker functions. Each torrent configuration owns its own info hash, listener port, data root, piece manager, counters, peer list, connection set, worker threads, and log. Keeping those objects separate prevents one swarm's availability or tracker results from leaking into another swarm. Around the same time, metainfo operations that had started to appear in several modules were moved into common/metainfo.py. The helper formalized loading, validation, total-size calculation, tracker URL extraction, and the SHA-1 of the encoded info dictionary. This refactoring happened after the network path existed, because by then it was clear which metainfo operations were truly shared and which belonged to the peer or piece manager. It reduced duplicated torrent-identity logic without changing the peer-wire behavior that had already been tested.

Generating metainfo and dealing with exact-byte mismatches. We then added torrent_generator.py so the demonstration metainfo could be regenerated directly from files in shared_files. The generator reads the exact source bytes, divides them into pieces, hashes every piece, concatenates the SHA-1 digests, builds announce and info, and writes the bencoded torrent. This step revealed an issue that was easy to miss when looking only at text content: torrent identity depends on exact bytes. A trailing newline in alphabet.txt changed its length and all hashes derived from it, so the source and torrent had to be brought back into sync and regenerated. We also encountered the line-ending difference for time_machine.txt: the Ubuntu LF version and a Windows CRLF version can display nearly the same text while producing different byte counts and SHA-1 values. The project instructions therefore include regeneration on Windows when line endings have changed. The JPEG does not have that text-line-ending problem, which is one reason it was used for the main transfer capture.

Checking each stage before extending it. We did not wait until the entire peer was finished to run the first complete test. Each subsystem was exercised when it became a dependency for the next one. Bencode and metainfo checks came before HTTP state, tracker lifecycle checks came before peer discovery, handshake and message parsers were checked before piece transfer, and file mapping and corruption handling were checked before received network bytes were committed to disk. Later checks covered the interaction between these parts: a real block download, the downloader-to-seeder transition, stale tracker cleanup, multiple torrent sessions, and multi-peer continuation. This incremental approach was useful because network failures often appear far from their actual source. If a final download is wrong, the cause may be an info-hash mismatch, an incorrect piece boundary, a stale reservation, a framing error, or a storage write. Keeping the previous layer stable narrowed the set of places that had to be inspected when a new stage failed.

Finishing with end-to-end runs and packet observation. After the protocol path was stable, we ran the two full scenarios that are reported in Section 11. The first used network.torrent with a seeder on port 6881 and a downloader on 6882. It exercised tracker discovery, direct TCP connection, handshake, availability exchange, request and piece messages, SHA-1 verification, file storage, completion, and continued keep-alive activity. The second used time_machine.torrent with three peers: peer 6884 first downloaded from 6883, the original seeder was stopped, and peer 6885 then downloaded from 6884. That run forced the implementation to use a completed downloader as a later upload source rather than

relying on the original copy. Wireshark was used after functional transfer was already working so packet captures could be interpreted against known application behavior. The tracker HTTP exchange, peer handshakes, framed control traffic, binary file data, full-duplex stream direction, and TCP reassembly were then matched with the event logs instead of being used as a substitute for application-level verification.

Closing the implementation cycle. With the end-to-end behavior stable, the remaining work was to make the project reproducible: keep the module responsibilities clear, document startup order and ports, record the timing values and line-ending warning, and preserve the logs and packet observations. The final implementation therefore reflects the order in which the dependencies became usable rather than a single pass in which all modules appeared fully formed. Encoding enabled torrent identity, tracker state enabled discovery, framing enabled reliable TCP messages, connection state enabled flow control, the piece manager made integrity decisions independent of sockets, and the concurrency work allowed those parts to operate at the same time. The later fixes to tracker lifecycle, unchoke state, socket shutdown, and metainfo byte consistency were kept as part of the final design because each one came from an actual integration problem encountered while connecting the subsystems.

5. Metainfo and Torrent Identity

5.1 Required structure

A .torrent file is a bencoded dictionary. The announce field contains the tracker URL. The info dictionary contains the file name or multi-file description, piece length, and a byte string formed by concatenating one 20-byte SHA-1 digest per piece.

Torrent identity: $\text{info_hash} = \text{SHA-1}(\text{bencode}(\text{info dictionary}))$. Dictionary keys are bencoded in sorted byte order so every peer calculates the same 20-byte identity.

For a single-file torrent, total size is the length value. For a multi-file torrent, total size is the sum of every file length. The number of expected pieces is $\text{ceiling}(\text{total size} / \text{piece length})$. The last piece may be shorter than the normal piece length.

5.2 Supplied torrents

Metainfo	Content	Bytes	Piece length	Actual piece sizes	Pieces
alphabet.torrent	alphabet.txt	26	10	10,10,6	3
network.torrent	network.jpg	48,932	262,144	48932	1
time_machine.torrent	time_machine.txt	495	262,144	495	1

Table 2 - Metainfo supplied with the project.

The generator scans ordinary files in `shared_files` and writes matching .torrent files into metainfo. Small alphabet data uses a short piece length to demonstrate multiple pieces; larger demonstration files use 256 KiB. The three torrent files also remain in separate swarms because their info hashes differ.

The names, sizes, piece counts, and hashes in Table 2 are demonstration inputs rather than hard-coded protocol values. The metainfo reader derives them from any valid bencoded single-file or multi-file info

dictionary, so replacing the supplied torrent files changes the runtime torrent identity and layout without requiring source-code changes.

6. Tracker Design

6.1 Announce processing

The tracker exposes a multithreaded HTTP GET endpoint at /announce. Each request is parsed as raw percent-decoded bytes where needed, validated, and applied to the swarm associated with its info hash. Malformed requests receive a bencoded failure reason rather than being added to state.

- **Torrent and peer identity.** info_hash and peer_id must each contain exactly 20 bytes. The info hash selects one swarm, while the peer ID distinguishes records inside that swarm.
- **Listening endpoint.** port must be between 1 and 65535 and identifies the peer's incoming TCP listener. The tracker takes the peer IP address from the HTTP connection rather than trusting a query-supplied address.
- **Transfer counters.** uploaded, downloaded, and left are non-negative byte counts. A left value of zero identifies a peer that owns all verified content and is counted as complete.
- **Lifecycle event.** event may be started, completed, stopped, or omitted for an ordinary periodic update. Unknown event values produce a bencoded failure response.

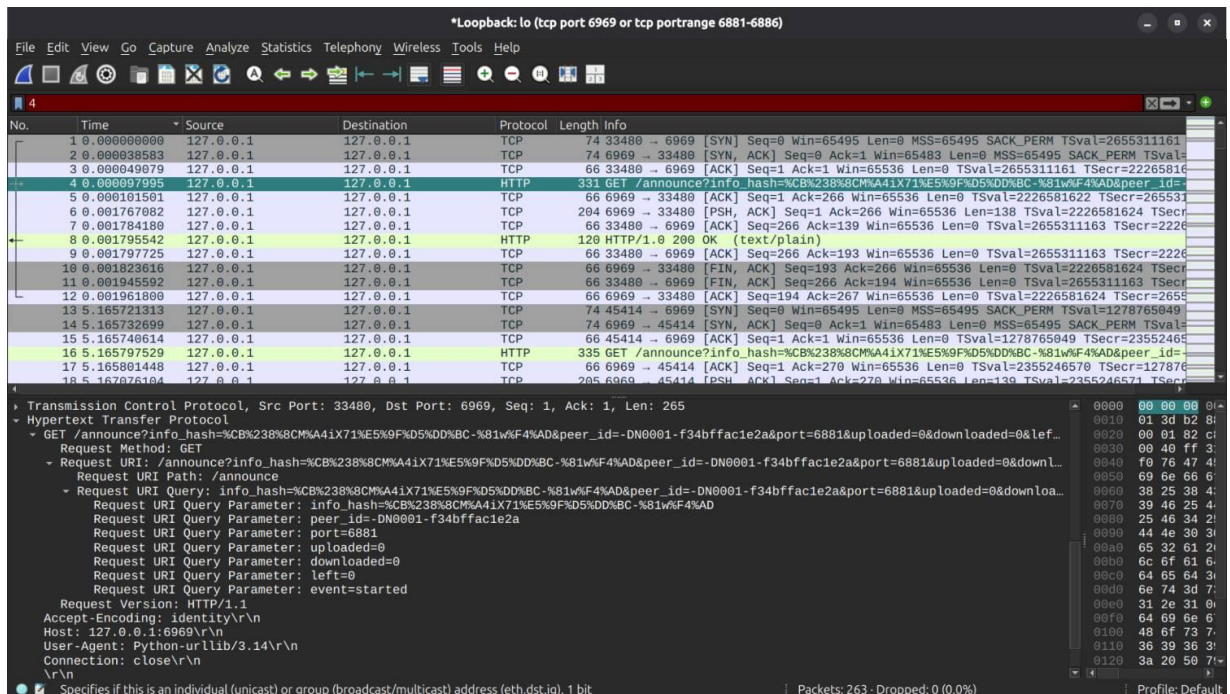


Figure 2 - Captured started announce to tracker port 6969. Wireshark decodes info_hash, peer_id, port=6881, byte counters, left, and event=started.

The packet list shows an HTTP connection from temporary source port 33480 to the fixed tracker port 6969. Inside the query, port=6881 advertises the peer's separate listening service. The two values are not expected to match: 33480 exists only for that outgoing HTTP connection, whereas 6881 remains

open for incoming peer-wire connections. Wireshark also displays percent-encoded info_hash bytes because a torrent identity is arbitrary binary data rather than ordinary text.

6.2 Response and lifecycle

A successful response contains interval, complete, incomplete, and peers. The tracker excludes the requesting peer from its returned list. Each peer entry contains peer_id, ip, and port. If validation fails, the response contains failure reason instead and does not mix that error with success fields.

- **started.** Creates a new peer record or refreshes an existing one. The peer's first announce for a session includes this event, as required by the tracker protocol.
- **Periodic update.** An announce with no event refreshes byte counters and last-seen time while the peer remains active.
- **completed.** Refreshes the record with left=0. The peer remains in the swarm but moves from the incomplete count to the complete count so that it can continue seeding.
- **stopped.** Removes the peer record immediately and deletes the swarm container if no peers remain.
- **Timeout cleanup.** A peer that has not announced for more than 90 seconds is removed during cleanup before a current response is built.

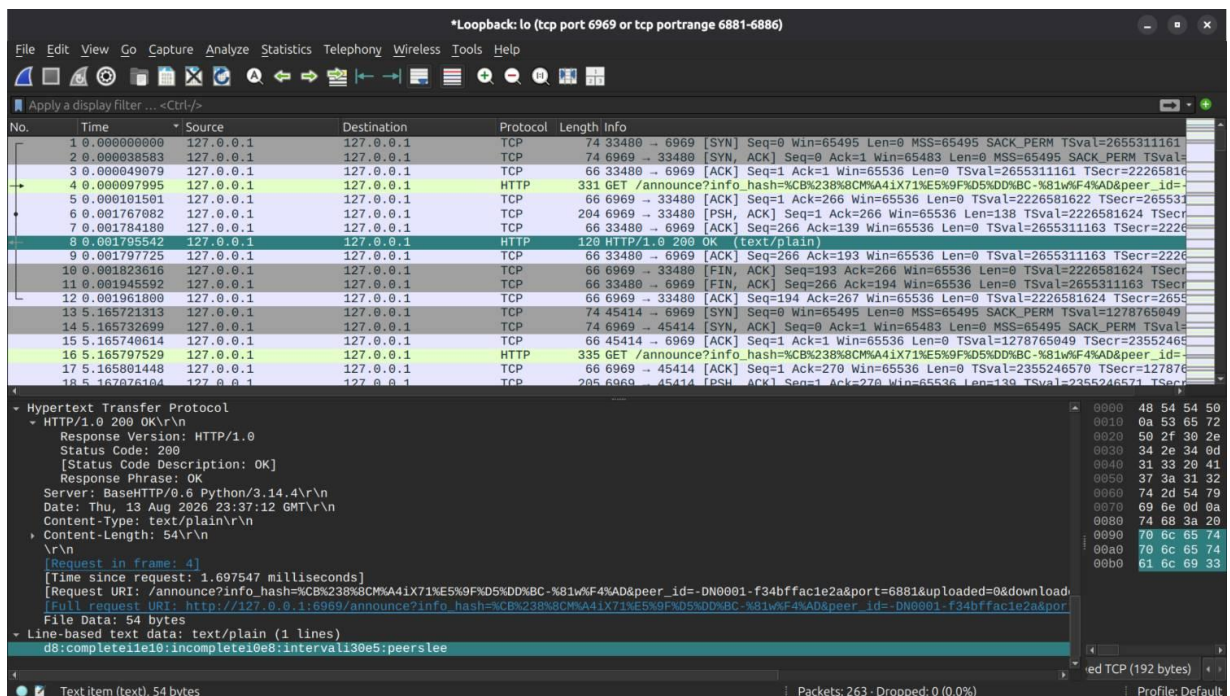


Figure 3 - HTTP 200 tracker response with the bencoded interval, complete, incomplete, and peers fields.

HTTP status 200 confirms that the HTTP exchange succeeded, but it is not by itself the application result. The peer must bdecode the response body, check whether failure reason is present, and then read interval, complete, incomplete, and peers. The selected capture contains the bencoded success fields; an empty peers list simply means that no other peer was available to return for that announce at that instant. The requesting peer is intentionally excluded from its own list.

The tracker advertises a 30-second reannounce interval. A 90-second peer timeout allows three missed intervals before stale removal. Monotonic time is used for timeout calculation so changes to the system wall clock do not incorrectly age peers.

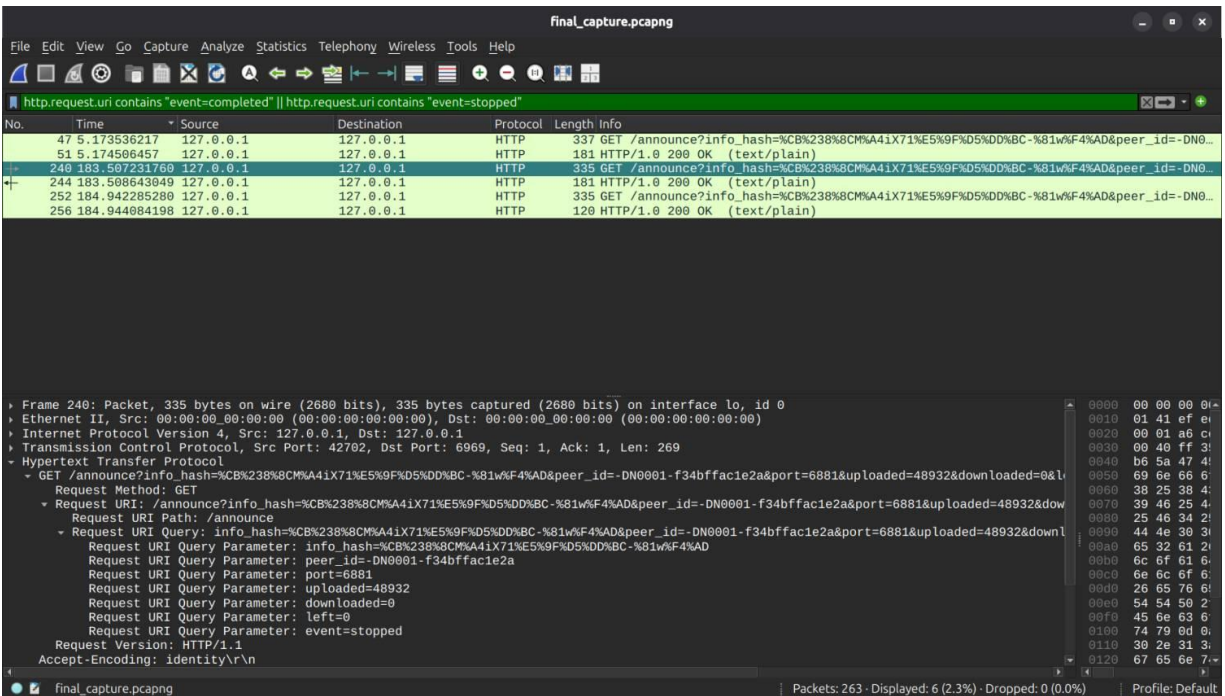


Figure 4 - Captured stopped announce. The peer reports its final counters and left=0 before the tracker removes its registration.

The stopped capture confirms that lifecycle state is carried as an ordinary announce parameter rather than a separate control connection. The request still includes the torrent identity, peer identity, listening port, and final counters, giving the tracker enough information to remove exactly one record from exactly one swarm. The response is built after removal and therefore reflects the remaining peers.

7. Peer Design and Peer-Wire Protocol

7.1 Peer lifecycle

1. Load and validate metainfo; calculate the 20-byte info hash and total torrent size.
2. Scan the data directory and mark only pieces whose stored bytes match the metainfo SHA-1 hash.
3. Start the TCP listener before announcing, so endpoints returned by the tracker are already reachable.
4. Announce started, process the returned peer list, and connect to new endpoints.
5. Complete the handshake and exchange bitfield/HAVE state before requesting data.
6. Request blocks only while unchoked; verify a complete piece before saving it and advertising HAVE.
7. Announce completed when left becomes zero; continue serving verified data as a seeder.
8. On shutdown, announce stopped and close listener and peer connections gracefully.

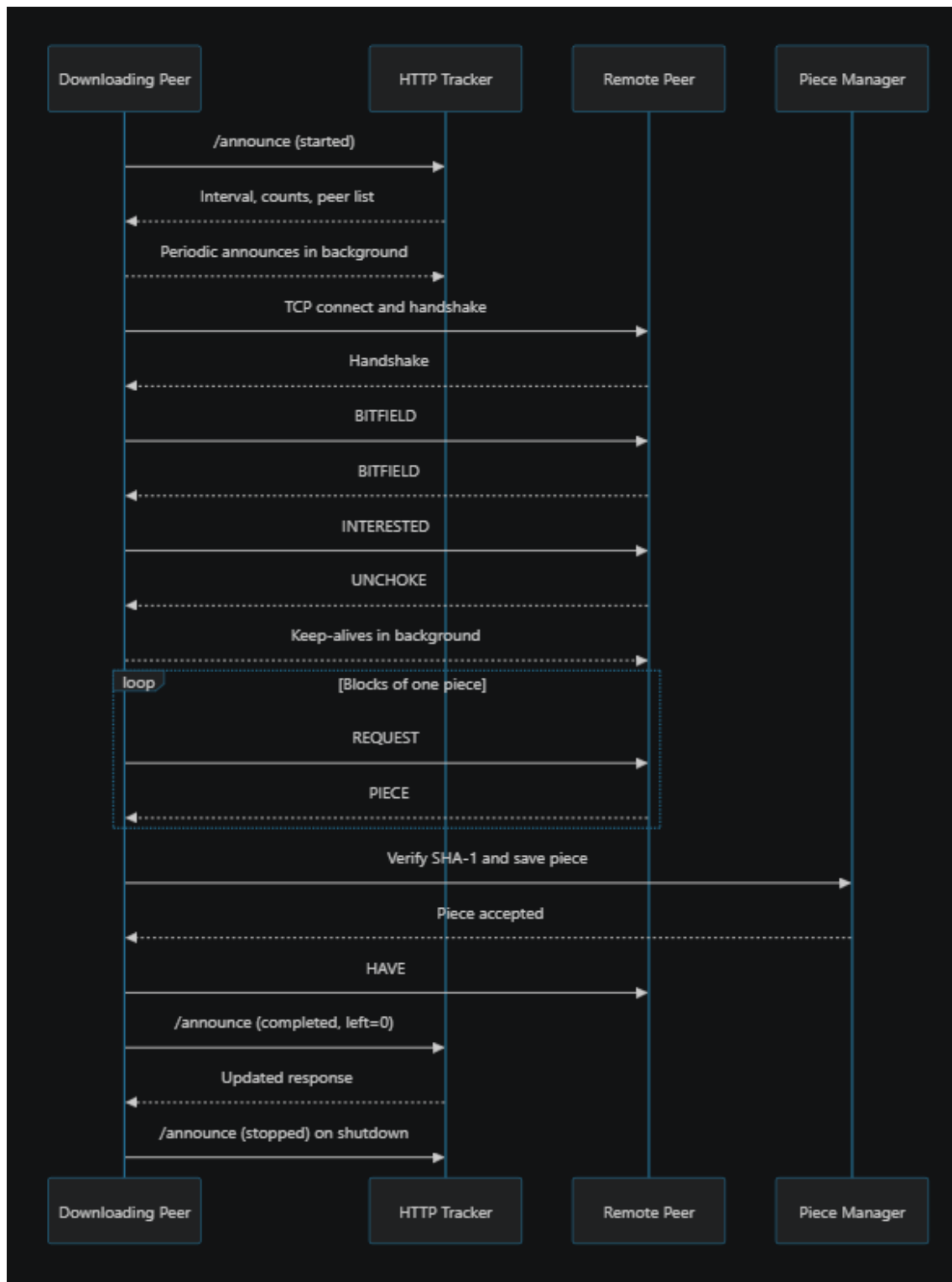


Figure 5 - Main tracker and peer-to-peer message sequence.

7.2 Handshake and message framing

The handshake is exactly 68 bytes: a one-byte protocol-string length, the 19-byte text “BitTorrent protocol”, eight reserved bytes, the 20-byte info hash, and the 20-byte peer ID. Both sides reject a connection if the handshake is malformed or if the info hash does not match the local torrent.

After the handshake, messages are framed by a four-byte unsigned big-endian length prefix. A zero length is keep-alive. Otherwise the next byte is the message ID and the remaining bytes are its payload. Strict payload-length and index/range checks are applied before a message changes peer state.

- **Keep-alive.** A zero-length frame carries no message ID and keeps an otherwise idle TCP connection observable and active.
- **CHOKE and UNCHOKER (IDs 0 and 1).** Control whether the remote peer is currently permitted to request file blocks.
- **INTERESTED and NOT_INTERESTED (IDs 2 and 3).** Indicate whether the remote availability information contains content that the sender still needs.
- **HAVE and BITFIELD (IDs 4 and 5).** Advertise one newly verified piece or the complete set of currently verified pieces. Bitfield bits are stored most-significant-bit first.
- **REQUEST (ID 6).** Names the piece index, byte offset within that piece, and requested block length.
- **PIECE (ID 7).** Returns the piece index, byte offset, and requested binary block data.

7.3 Transfer policy and connection state

Each connection tracks local and remote choking and interest state, the remote piece set, a current piece reservation, and one outstanding block request. A peer sends INTERESTED only when the remote bitfield contains a locally missing piece. An interested peer is unchoked in this design.

Pieces are reserved so two local connections do not download the same piece simultaneously. Blocks are requested sequentially with a maximum size of 16 KiB. This conservative one-request-at-a-time strategy keeps the request state straightforward and avoids overlapping requests.

7.4 Periodic tracker refresh, pings, and torrent pool

The peer follows the interval returned by the tracker and refreshes its swarm membership periodically. A separate keep-alive loop sends a zero-length peer message every 15 seconds on active peer connections. Tracker refresh and peer keep-alive are independent because they operate on different connections and solve different liveness problems.

The torrent pool starts one full peer session per configuration. Each session has its own metainfo, info hash, listening port, data root, counters, and worker threads. Consequently, one process can participate in multiple swarms without mixing piece state or peer lists.

8. Piece Management and File Integrity

8.1 Verification before commitment

A received block is accepted only if it exactly matches the outstanding request's piece index, start offset, and expected length. Blocks are assembled in memory until the complete piece is present. The piece is then hashed and compared with the corresponding 20-byte digest in the metainfo.

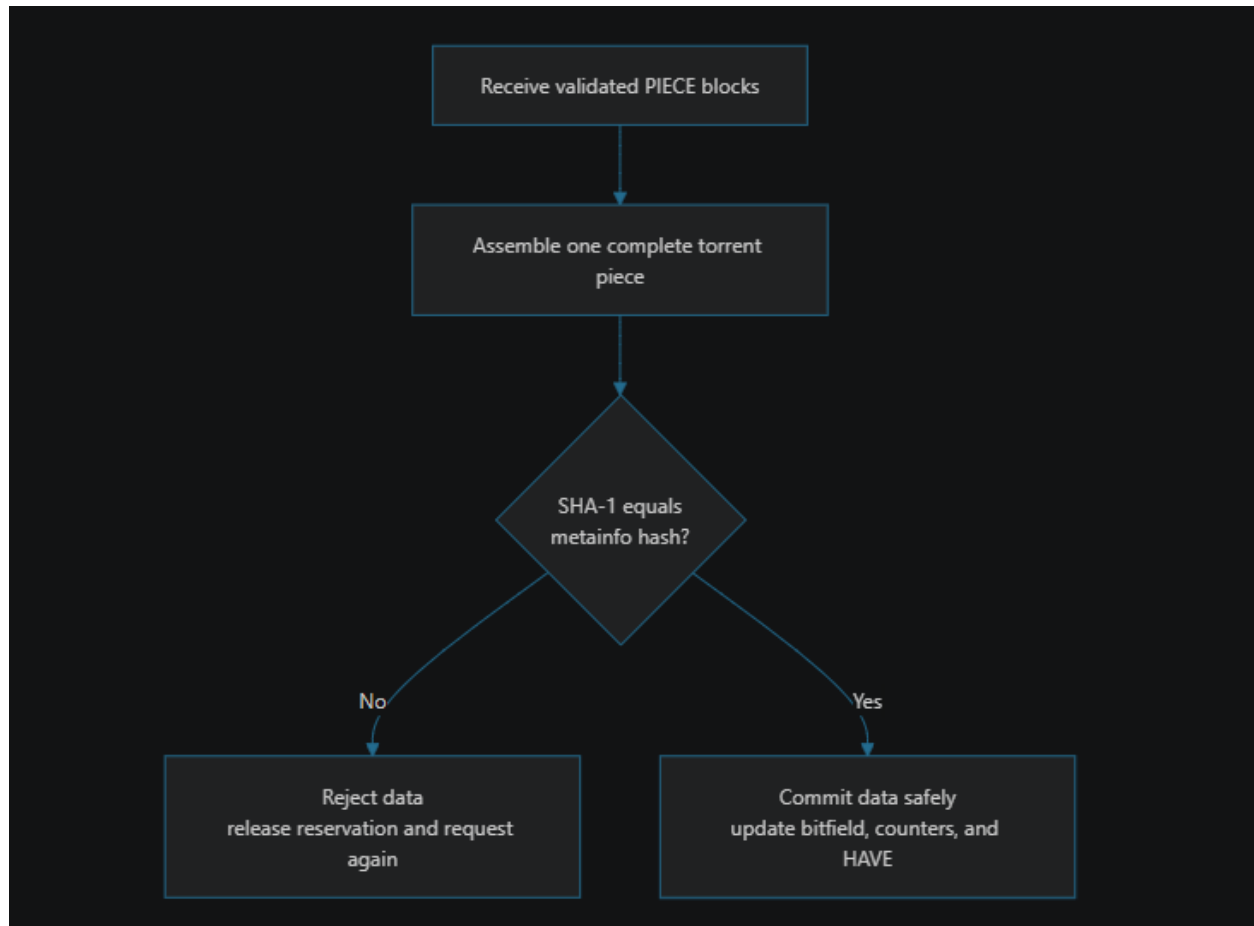


Figure 6 - Piece verification and storage decision.

Only a verified piece is written to storage, counted as downloaded, included in the local bitfield, or advertised with HAVE. If the hash does not match, the data is rejected and its reservation is released for a later request. Existing files are also scanned at startup, so corrupted stored data is not treated as available.

8.2 File mapping

The piece manager treats torrent content as one logical byte range. For a single-file torrent this range maps directly to one file. For a multi-file torrent, file offsets are calculated cumulatively; a piece that crosses a boundary is divided into the correct per-file writes. Final file sizes are normalized to the lengths declared in metainfo.

8.3 Path safety

Metainfo paths are validated before use. Absolute paths, empty components, dot or parent-directory components, embedded separators, and null bytes are rejected. This prevents a torrent from escaping the selected data root. Directories are created only beneath that root.

9. Logging and Observability

9.1 Required log format

The tracker writes logs/tracker.log and each peer writes a port-specific file such as logs/peer_6884.log. A process-wide lock serializes log updates so concurrent threads do not interleave entries. The file header preserves the original creation time and refreshes the last-modified time.

```
File Name: peer_6884.log
Creation Date/Time: 2026-08-14 05:00:52
Last Modified Date/Time: 2026-08-14 05:02:52

--- EVENTS ---
Event Type: PEER_STARTED
Date/Time: 2026-08-14 05:00:52
Description: Listening on port 6884
-----
Event Type: TRACKER_RESPONSE
Date/Time: 2026-08-14 05:00:52
Description: Received 1 peers
-----
Event Type: PEER_HANDSHAKE
Date/Time: 2026-08-14 05:00:52
Description: Handshake completed with ('127.0.0.1', 6883)
-----
Event Type: BITFIELD_SENT
Date/Time: 2026-08-14 05:00:52
Description: Sent bitfield 00 to ('127.0.0.1', 6883)
-----
Event Type: BITFIELD_RECEIVED
Date/Time: 2026-08-14 05:00:52
Description: Peer ('127.0.0.1', 6883) has pieces [0]
-----
Event Type: INTERESTED_SENT
Date/Time: 2026-08-14 05:00:52
Description: Interested in pieces from ('127.0.0.1', 6883)
-----
Event Type: UNCHOKE_RECEIVED
Date/Time: 2026-08-14 05:00:52
Description: Peer ('127.0.0.1', 6883) unchoked us
-----
Event Type: REQUEST_SENT
Date/Time: 2026-08-14 05:00:52
Description: Requested piece 0 bytes 0:495 from ('127.0.0.1', 6883)
-----
Event Type: PIECE_RECEIVED
Date/Time: 2026-08-14 05:00:52
Description: Received piece 0 bytes 0:495 from ('127.0.0.1', 6883)
-----
Event Type: PIECE_VERIFIED
Date/Time: 2026-08-14 05:00:52
Description: Piece 0 passed SHA-1 verification
-----
Event Type: HAVE_SENT
Date/Time: 2026-08-14 05:00:52
Description: Sent piece 0 to ('127.0.0.1', 6883)
-----
Event Type: NOT_INTERESTED_SENT
Date/Time: 2026-08-14 05:00:52
Description: No longer interested in ('127.0.0.1', 6883)
-----
Event Type: DOWNLOAD_COMPLETED
Date/Time: 2026-08-14 05:00:52
Description: All pieces were downloaded and verified
```

The header fields and the three fields in every event entry match the required logging structure. Descriptions include the relevant port, remote endpoint, piece index, peer count, or error where useful.

9.2 Logged operational events

- **Tracker coordination.** TRACKER_STARTED, PEER_STARTED, PEER_UPDATE, PEER_COMPLETED, PEER_STOPPED, PEER_TIMEOUT, TRACKER_RESPONSE, and TRACKER_ERROR expose swarm registration and accounting.
- **Peer lifecycle.** PEER_STARTED, TRACKER_RESPONSE, PEER_CONNECTED, PEER_HANDSHAKE, PEER_DISCONNECTED, and PEER_STOPPED show how a session enters and leaves the system.
- **Availability and flow control.** BITFIELD and HAVE events explain which pieces are advertised, while INTERESTED, NOT_INTERESTED, CHOKE, and UNCHOKE explain whether transfer is currently allowed.
- **Data transfer.** REQUEST, PIECE, PIECE_VERIFIED, and DOWNLOAD_COMPLETED connect individual block exchanges to a verified application result.
- **Liveness and errors.** PEER_PING_SENT, PEER_PING_RECEIVED, PEER_ERROR, and TRACKER_ERROR identify idle connection activity and failures that need investigation.

The categories form a chronological explanation of system behavior. For example, tracker discovery is followed by connection and handshake events, availability and flow-control decisions, block transfer, piece verification, completion, and later keep-alive activity. The log therefore complements Wireshark: the capture records the bytes on the wire, while the log records the system's interpretation of those bytes.

10. Wireshark Capture and TCP Interpretation

10.1 Capture method

Wireshark captured the Ubuntu loopback interface `lo` while the tracker and peers ran on the same virtual machine. The capture filter was `tcp port 6969 or tcp portrange 6881-6886`. Tracker traffic is identified by TCP port 6969 and HTTP decoding; peer-wire traffic is identified by the configured peer listener ports.

The capture includes the fixed service ports and the operating system's temporary client ports. A packet to destination port 6969 belongs to the tracker HTTP service; a connection to ports such as 6881 or 6882 belongs to a peer listener. The temporary source ports are connection endpoints chosen by the operating system and do not replace the ports advertised by the services.

10.2 Tracker HTTP analysis

Figure 2 records a complete started announce. The request path is `/announce` and Wireshark expands the query into `info_hash`, `peer_id`, `port`, `uploaded`, `downloaded`, `left`, and `event`. The 20-byte identifiers are percent-encoded because they contain arbitrary binary values. The advertised port=6881 is the peer's listener; the temporary TCP source port shown above it belongs only to this HTTP request.

Figure 3 records the reverse direction. The tracker returns HTTP 200 with a text/plain body whose bytes form a bencoded dictionary. The complete and incomplete values describe the current swarm, interval tells the peer when to announce again, and peers supplies direct connection endpoints. The body, rather than the HTTP status alone, determines whether the announce succeeded at the application level.

10.3 Peer-wire stream analysis

Figure 7 is a Follow TCP Stream view of one peer connection. Red and blue represent the two directions of the same full-duplex TCP stream. Each direction begins with `0x13`, which is decimal 19, followed by the text BitTorrent protocol, eight reserved bytes, the same 20-byte info hash, and a different 20-byte peer ID. Matching info hashes indicate that both endpoints accepted the connection for the same torrent.

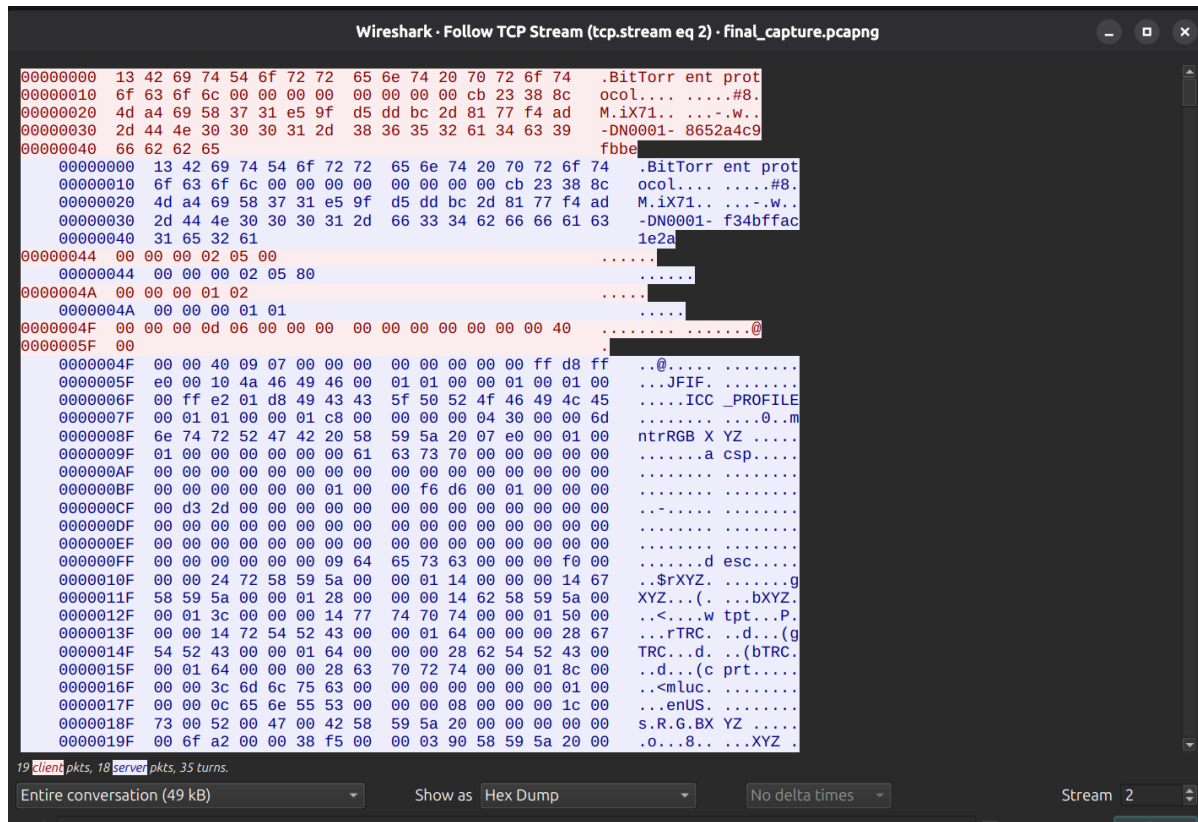


Figure 7 - Follow TCP Stream in hexadecimal form. Both 68-byte handshakes begin with “BitTorrent protocol”, followed by control frames and JPEG bytes beginning with JFIF.

The bytes following the handshakes show the application state change and transfer. For example, 00 00 00 02 05 80 is a two-byte message body with ID 5 and a bitfield whose first bit is set. Interest and unchoke frames then permit downloading. A normal REQUEST body is 13 bytes and names the piece, begin offset, and block length. A full 16 KiB PIECE block has a length prefix of 0x4009 (16,393 bytes): one message-ID byte, two four-byte fields, and 16,384 data bytes. The visible FF D8 FF E0 and JFIF values are the beginning of the transferred JPEG content, confirming that file data traveled directly between peers.

```

peers:
- peer: 0
  host: 127.0.0.1
  port: 58474
- peer: 1
  host: 127.0.0.1
  port: 6881
packets:
- packet: 28
  peer: 0
  index: 0
  timestamp: 1786664238.119733885
  data: !!binary |
    E0JpdFRvcnJlbnQgcHJvdG9jb2wAAAAAAAAAAMsj0IxNpGlyNzHln9Xdvc2Bd/StLUROMDAwMS04
    NjUyYTRjOWZiYmU=
- packet: 30
  peer: 1
  index: 0
  timestamp: 1786664238.120102150
  data: !!binary |
    E0JpdFRvcnJlbnQgcHJvdG9jb2wAAAAAAAAAAMsj0IxNpGlyNzHln9Xdvc2Bd/StLUROMDAwMS1m
    MzRiZmZhYzFlMmE=
- packet: 32
  peer: 0
  index: 1
  timestamp: 1786664238.120483850
  data: !!binary |
    AAAAagUA
- packet: 33
  peer: 1
  index: 1
  timestamp: 1786664238.120847498
  data: !!binary |
    AAAAagWA
- packet: 34
  peer: 0
  index: 2
  timestamp: 1786664238.121338947

```

Packet 33: 19 Client pkts, 18 Server pkts, 35 turns. Click to select.

Entire conversation (49 kB) Show as: YAML No delta times Stream: 2

Figure 8 - The same peer TCP conversation in YAML form, showing both endpoints, packet direction, timestamps, and binary stream data.

Figure 8 presents the same full-duplex connection in Wireshark's YAML view. Peer 0 uses the temporary client port 58474, while peer 1 is the fixed listener on port 6881. Each entry identifies its direction, capture-frame number, position within that direction, timestamp, and binary data. Wireshark displays the binary bytes with YAML's `!!binary` representation, so the encoded text is a view of the captured bytes rather than the original file format. The alternating peer values confirm two-way communication, but the listed packet records are still observations of one reassembled TCP byte stream and are not used as peer-message boundaries.

10.4 TCP stream boundaries and reassembly

A torrent piece, a peer message, and a TCP segment are different units. A piece is an application-level file unit protected by one SHA-1 hash. A request transfers that piece in one or more blocks. TCP exposes an ordered byte stream and may divide one application message among several segments or combine several messages into one segment.

TCP stream handling. Peer-wire communication is treated as a byte stream rather than as individual TCP packets. The peer first reads the four-byte message-length prefix and then continues receiving until the complete announced body is available. As a result, peer-wire parsing does not depend on the packet boundaries or packet counts visible in Wireshark.

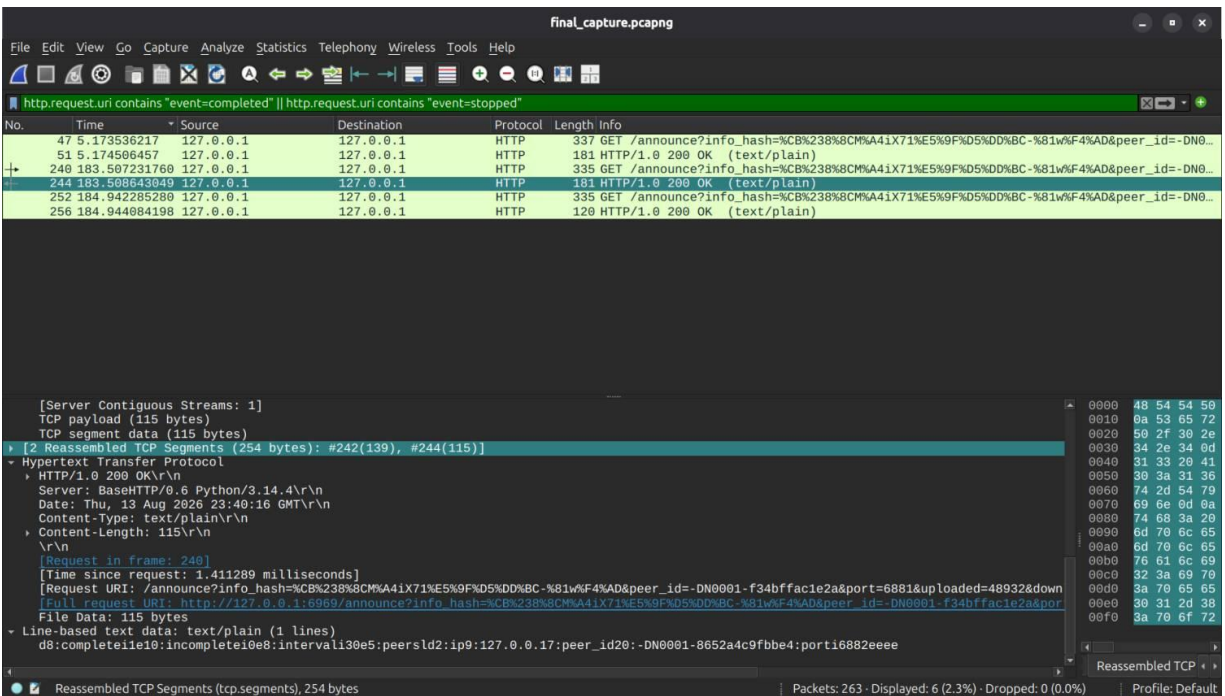


Figure 9 - Wireshark reassembles one tracker response from two TCP segments before decoding the complete HTTP and bencoded payload.

Figure 9 makes this distinction visible: Wireshark reports that two TCP segments were reassembled before the complete HTTP response could be decoded. The tracker client receives HTTP through the standard-library HTTP reader, which reconstructs the response stream. The peer protocol uses exact-length reads: it first collects four bytes for the length prefix and then continues until the full announced body is available. Either approach remains correct when the network divides or combines TCP segments differently.

11. End-to-End Verification Results

11.1 Two-peer JPEG transfer

The first recorded scenario used network.torrent. Peer 6881 started with shared_files/network.jpg and peer 6882 started with an empty downloads/wireshark_run directory. The downloader discovered the seeder through the tracker, completed the handshake and control exchange, received the 48,932-byte piece, verified it, saved it, and continued exchanging keep-alive messages.

Although network.jpg is smaller than the 262,144-byte piece length and therefore forms one torrent piece, it is still transferred as several bounded blocks of at most 16 KiB. This distinction matters: the block requests control transport progress, while the single piece hash is the final integrity decision. The peer announces completed only after all of those blocks have been assembled and the whole 48,932-byte piece has passed SHA-1 verification.

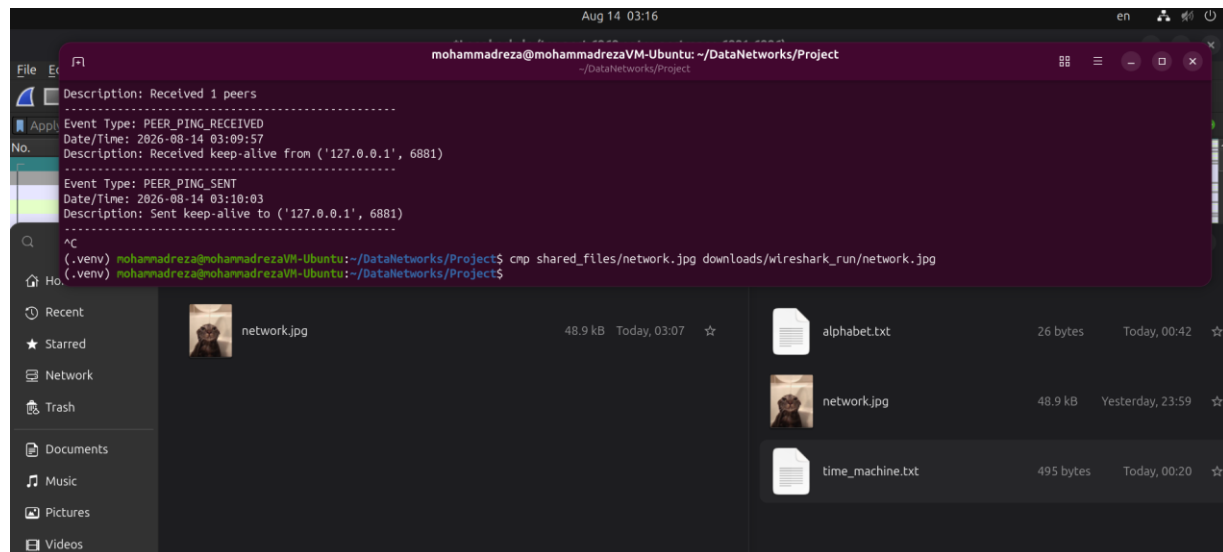


Figure 10 - The source and downloaded JPEG are visible, and cmp returns silently, which means the files are byte-for-byte identical.

The downloaded image opening successfully is useful visual evidence, but it is not a sufficient integrity test because some damaged image files may still be partially displayed. The silent cmp result is stronger: cmp prints nothing only when every byte and the total file length are identical. This external comparison agrees with the internal piece-hash decision recorded before DOWNLOAD_COMPLETED.

11.2 Three-peer swarm with a different torrent

The second scenario used time_machine.torrent rather than the JPEG torrent. Peer 6883 was the original seeder. Peer 6884 downloaded and verified the file, becoming a seeder. After the original seeder stopped, peer 6885 joined and completed its download from peer 6884. Peer 6884 therefore remained available as an upload source after completing its own download.

- **First transfer stage.** Peer 6883 advertises the complete time_machine torrent. Peer 6884 discovers it through the tracker, downloads the content, verifies the piece, and changes its left counter to zero.

- **Seeder-transition stage.** After peer 6883 stops, peer 6884 remains registered as complete. Peer 6885 then discovers peer 6884 and downloads the same verified content from that former downloader.

```

mohammadreza@mohammadrezaVM-Ubuntu: ~/DataNetworks/Project
(.venv) mohammadreza@mohammadrezaVM-Ubuntu:~/DataNetworks/Project$ PYTHONPATH=.
python -m tracker.tracker
Tracker running at http://127.0.0.1:6969/announce

mohammadreza@mohammadrezaVM-Ubuntu: ~/DataNetworks/Project
(.venv) mohammadreza@mohammadrezaVM-Ubuntu:~/DataNetworks/Project$ tail -f logs/peer_6884.log
Description: No longer interested in ('127.0.0.1', 6883)
Event Type: DOWNLOAD_COMPLETED
Date/Time: 2026-08-14 05:00:52
Description: All pieces were downloaded and verified
Event Type: PEER_PING_RECEIVED
Date/Time: 2026-08-14 05:00:56
Description: Received keep-alive from ('127.0.0.1', 6883)
Event Type: PEER_PING_SENT
Date/Time: 2026-08-14 05:01:07
Description: Sent keep-alive to ('127.0.0.1', 6883)
Event Type: PEER_PING_RECEIVED
Date/Time: 2026-08-14 05:01:11
Description: Received keep-alive from ('127.0.0.1', 6883)
^C
(.venv) mohammadreza@mohammadrezaVM-Ubuntu:~/DataNetworks/Project$ tail -f logs/peer_6885.log
Description: No longer interested in ('127.0.0.1', 6884)
Event Type: DOWNLOAD_COMPLETED
Date/Time: 2026-08-14 05:01:51
Description: All pieces were downloaded and verified
Event Type: PEER_PING_RECEIVED
Date/Time: 2026-08-14 05:01:52
Description: Received keep-alive from ('127.0.0.1', 6884)
Event Type: PEER_PING_SENT
Date/Time: 2026-08-14 05:02:06
Description: Sent keep-alive to ('127.0.0.1', 6884)
Event Type: PEER_PING_RECEIVED
Date/Time: 2026-08-14 05:02:07
Description: Received keep-alive from ('127.0.0.1', 6884)
Event Type: PEER_PING_SENT
Date/Time: 2026-08-14 05:02:21
Description: Sent keep-alive to ('127.0.0.1', 6884)

```

Figure 11 - Three-peer time_machine torrent run. The logs show completion at peer 6884 and later at peer 6885, with keep-alive traffic to the peer that supplied each copy.

```

mohammadreza@mohammadrezaVM-Ubuntu: ~/DataNetworks/Project
(.venv) mohammadreza@mohammadrezaVM-Ubuntu:~/DataNetworks/Project$ cmp shared_files/time_machine.txt downloads/time_swarm_peer2/time_machine.txt
cmp shared_files/time_machine.txt downloads/time_swarm_peer3/time_machine.txt
(.venv) mohammadreza@mohammadrezaVM-Ubuntu:~/DataNetworks/Project$

Name      Size      Modified
time_machine.txt  495 bytes  Today, 05:01
time_machine.txt  495 bytes  Today, 05:00

```

Figure 12 - Both downloaded time_machine.txt files are shown, and both cmp commands return no differences from the source.

The two silent comparisons confirm that both downloaded copies match the original 495-byte source. More importantly, the second copy was produced after the original seeder had left. This is evidence of actual peer-to-peer redistribution: the tracker supplied discovery information, but peer 6884, not the tracker, provided the file data to peer 6885.

11.3 Multi-piece transfer

An additional execution used `alphabet.torrent` to observe the transfer path across more than one piece. The 26-byte `alphabet.txt` file uses a 10-byte piece length, giving three pieces of 10, 10, and 6 bytes. Peer 6886 started with the source file and peer 6887 downloaded into `downloads/alphabet_peer_6887`. The downloader processed pieces 0, 1, and 2 separately; each piece passed SHA-1 verification and was followed by a HAVE update. `DOWNLOAD_COMPLETED` was recorded after the third piece.

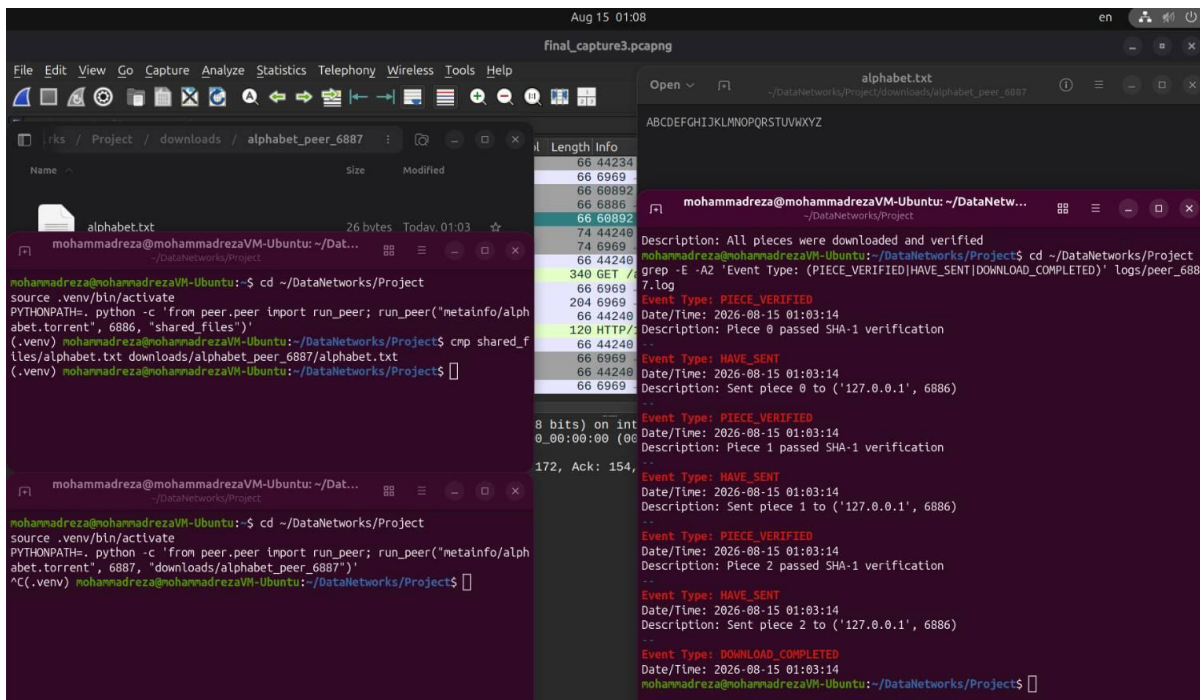


Figure 13 - Multi-piece alphabet transfer. The log shows separate verification and HAVE events for pieces 0, 1, and 2, followed by `DOWNLOAD_COMPLETED`; the silent `cmp` command shows no difference from the source.

The silent comparison was run with `cmp shared_files/alphabet.txt downloads/alphabet_peer_6887/alphabet.txt`. No output indicates that the downloaded 26-byte file matches the source byte for byte. Unlike the earlier recorded transfers, this run passes through piece selection, verification, and HAVE advertisement three times before completion.

11.4 Result summary

- **Integrity result.** The 48,932-byte JPEG and both text downloads are byte-for-byte identical to their source files.
- **Multi-piece result.** The 26-byte alphabet file was transferred as three pieces of 10, 10, and 6 bytes; pieces 0, 1, and 2 were separately verified and advertised with HAVE before `DOWNLOAD_COMPLETED`.

- **Protocol result.** Tracker discovery, handshake, availability exchange, interest and unchoke, request and piece transfer, completion, and keep-alive behavior are visible across the logs and packet capture.
- **Seeder-transition result.** Peer 6884 successfully served a later downloader after obtaining and verifying its own copy from peer 6883.
- **Swarm-isolation result.** alphabet, network, and time_machine metainfo coexist with different info hashes, so tracker records and peer sessions remain separated by torrent identity.
- **Connection maintenance.** Periodic tracker updates refresh active peer records, while keep-alive frames provide periodic activity on active peer TCP connections.

12. Design Decisions

- **Localhost deployment.** The tracker and peers were run on the same Ubuntu virtual machine using 127.0.0.1. This avoids adding NAT, firewall, and routing configuration to the local setup while keeping communication over normal TCP sockets, HTTP connections, and separate service ports.
- **Threaded tracker.** Peers may announce at the same time, so ThreadingHTTPServer assigns a handler thread to each request. A lock protects the shared swarm dictionaries from overlapping updates.
- **Thirty-second announces and a ninety-second timeout.** Peers refresh their tracker state every 30 seconds. A peer is treated as stale after 90 seconds without an announce, allowing several missed updates before its record is removed.
- **Fifteen-second peer keep-alive.** Keep-alive frames are sent every 15 seconds on active peer connections. This provides periodic connection activity and allows completed peers to remain available for later requests.
- **Sixteen-kibibyte blocks.** Piece data is requested in blocks of at most 16 KiB. This allows larger pieces to be transferred incrementally while keeping request offsets and outstanding-block state relatively simple.
- **Verification before commitment.** Received blocks are assembled into a complete piece before the data affects local availability or storage. The complete piece is checked against its metainfo SHA-1 digest before being stored and advertised to other peers.
- **Simple piece selection.** Missing pieces are selected using a deterministic first-available policy, with one reservation and one outstanding block request per connection. More advanced strategies such as rarest-first selection and request pipelining are not part of the transfer policy used in this implementation.

These choices keep the peer and tracker behavior relatively small and traceable. The design favors clear state handling and direct observation over more advanced scheduling, persistence, and Internet-scale features.

13. Conclusion

The final system combines tracker-based peer discovery, direct TCP communication, piece-based file transfer, SHA-1 verification, and continued seeding within a single BitTorrent-style workflow. The tracker maintains swarm information and returns peer endpoints, while the peers handle the actual control messages and file data directly between one another.

The recorded executions included a JPEG transfer, a three-peer continuation scenario, and a multi-piece text transfer. Across these runs, downloaded data was stored after piece verification, completed peers remained available as upload sources, and separate torrents remained isolated through their different info hashes.

Structured event logs and Wireshark captures were used alongside the runtime behavior to follow tracker communication, peer-wire exchanges, file transfer, and TCP stream handling.